Algorithm 1 Top-Level Multi-Head Attention Accelerator Orchestration (`attention_core_top`)

Require: Matrices $Q, K, V \in \text{INT8}^{N \times d_k}$ per head stored in off-chip DDR4 memory;
Dyadic scaling parameters: Mantissa $M \in [0, 2^{18} - 1]$, Exponent $E \in [0, 31]$;
Softmax Exp-LUT parameters (1024×20-bit fixed-point coefficients in DDR4);
Constants: Sequence length $N = 256$, Head dimension $d_k \in \{16, 20, 24\}$, Number of heads $H = 4$;
AXI4-Stream bus width $W_{\text{bus}} \in \{64, 128\}$ bits;
Control flags: `start_pulse`, `update_lut_cfg`, `clear_buf`, `soft_reset`.
Ensure: Context output matrix $O \in \text{INT8}^{N \times d_k}$ per head written back to DDR4 memory;
Core status flag `DONE` asserted upon completion.

if `update_lut_cfg` is **true then**
    Load 1024-word Exp-LUT coefficients from DDR into True Dual-Port BRAM via AXI-Lite MMIO (Reg `0x40/0x44`)
end if
for each attention head $h \in [0, H)$ **do**
    DMA_MM2S_Transfer($Q[h], K[h], V[h]$) from DDR into on-chip Ping-Pong BRAM ($Q_{\text{MEM}}, K_{\text{MEM}}, V_{\text{MEM}}$)
    for each query token row $i \in [0, N)$ **do**
        $\mathbf{s}_{\text{raw}}[i, :] \leftarrow$ STAGE1_QKT_GEMM($Q_{\text{MEM}}[i, :], K_{\text{MEM}}, N, d_k$) ▷ *Call Algorithm 2*
        $\mathbf{s}_{\text{scaled}}[i, :] \leftarrow$ STAGE2_DYADIC_SCALER($\mathbf{s}_{\text{raw}}[i, :], M, E, N$) ▷ *Call Algorithm 3*
        $\mathbf{P}[i, :] \leftarrow$ STAGE3_HARDWARE_INTSOFTMAX($\mathbf{s}_{\text{scaled}}[i, :], N$) ▷ *Call Algorithm 4*
        $\mathbf{o}_i \leftarrow$ STAGE4_CONTEXT_GEMM($\mathbf{P}[i, :], V_{\text{MEM}}, N, d_k$) ▷ *Call Algorithm 5*
        Store token vector $O[i, :] \leftarrow \mathbf{o}_i$ into Egress Ping-Pong Buffer
    end for
    DMA_S2MM_Transfer(O) to DDR4 via AXI4-Stream Egress (W_{bus} beats, `tlast` asserted at final beat)
end for
Assert core `DONE` interrupt and update `STATUS` register (Reg `0x04`[1] $\leftarrow$ 1)

Algorithm 2 Stage 1 — Generalized QK^T GEMM Row Computation & Microarchitecture Mapping

Require: Query row vector $\mathbf{q}_i = Q[i, 0 : d_k - 1] \in \text{INT8}^{1 \times d_k}$;
Key matrix $K \in \text{INT8}^{N \times d_k}$ stored in on-chip Ping-Pong BRAM (K_{MEM});
Parameters: Sequence length $N = 256$, Head dimension $d_k \in \{16, 20, 24\}$, Bus width $W_{\text{bus}} \in \{64, 128\}$ bits;
Target Variant $\in$ {**Variant 1A**, **Variant 2A**, **Variant 3B**}.
Ensure: Raw Attention Score row vector $\mathbf{s}_{\text{raw}}[i, 0 : N - 1] \in \text{INT32}^{1 \times N}$.

▷ **Case 1: Variant 1A — Pure LUT Streaming with $N = 32$ Parallel Adder Tree (`pipelined_adder_tree`)**
if Target Variant == **Variant 1A then**
  Latch query row vector $\mathbf{q}_i$ into local query registers ▷ *Stationary across all N key tokens*
  for each key token index $j \in [0, N)$ **do**
    Fetch key token vector $K[j, 0 : d_k - 1]$ from BRAM ▷ *1 beat/token on 128-bit bus, zero-padded to 32B*
    for each channel $c \in [0, 32)$ **parallel do**
      $\text{prod}[c] \leftarrow (c < d_k)\ ?\ \big(\mathbf{q}_i[c] \times K[j, c]\big) : 0$ ▷ *32 parallel INT8 multipliers in LUT*
    end for
    $s_{\text{raw}}[i, j] \leftarrow \textsc{Pipelined_Adder_Tree_5_Stages}\big(\text{prod}[0 : 31]\big)$ ▷ *Throughput = 1 cycle ($II = 1$)*
  end for

▷ **Case 2: Variant 2A — Interleaved DSP-Packed Streaming (`dsp_packed_adder_tree`, 8 DSPs, Dual Trees)**
else if Target Variant == **Variant 2A then**
  Latch query row vector $\mathbf{q}_i$ into local query registers ▷ *Stationary across all key token pairs*
  $n_{\text{beats}} \leftarrow \lceil d_k/8 \rceil$ ▷ *Token sub-beats: 2 beats for $d_k = 16$; 3 beats for $d_k = 20, 24$*
  for each key token pair $p \in [0, N/2)$ **do** ▷ *Stream 128 pairs of Even ($j_e = 2p$) and Odd ($j_o = 2p + 1$) tokens*
    $j_e \leftarrow 2p, \quad j_o \leftarrow 2p + 1$
    Initialize beat accumulators: $\text{acc}_e \leftarrow 0, \quad \text{acc}_o \leftarrow 0$
    for each beat $b \in [0, n_{\text{beats}})$ **do**
      Fetch slices $\mathbf{q}_i[8b : 8b + 7]$, $K[j_e, 8b : 8b + 7]$, $K[j_o, 8b : 8b + 7]$ from BRAM
      for each DSP slice $d \in [0, 8)$ **parallel do** ▷ *8 DSP48E2 packed MAC units (27×18-bit)*
        $\text{prod}_e[d] \leftarrow \mathbf{q}_i[8b + d] \times K[j_e, 8b + d]$ ▷ *Port A × Port B (K_{even} product)*
        $\text{prod}_o[d] \leftarrow \mathbf{q}_i[8b + d] \times K[j_o, 8b + d]$ ▷ *Port A × Port C (K_{odd} product)*
      end for
      $\text{acc}_e \leftarrow \text{acc}_e + \textsc{Adder_Tree_3_Stages}(\text{prod}_e[0 : 7])$ ▷ *3-stage binary tree ($8 \to 4 \to 2 \to 1$)*
      $\text{acc}_o \leftarrow \text{acc}_o + \textsc{Adder_Tree_3_Stages}(\text{prod}_o[0 : 7])$ ▷ *Parallel tree for odd token*
    end for
    $\{s_{\text{raw}}[i, j_e],\ s_{\text{raw}}[i, j_o]\} \leftarrow \textsc{Zero_Bubble_Interleaver}(\text{acc}_e, \text{acc}_o)$ ▷ *Sustained throughput $II = 1$ cycle*
  end for

▷ **Case 3: Variant 3B — 2D Output-Stationary Systolic MAC Array (`systolic_mac_array`, 4×4 Grid, 16 DSPs)**
else if Target Variant == **Variant 3B then**
  Constants: Tile rows $T_{\text{row}} = 4$, Tile columns $T_{\text{col}} = 8$ (via 4×4 PE grid with dual-INT8 packing)
  for each column block $j_{\text{blk}} \in [0, N)$ with step T_{col} **do**
    $c_{\text{blk_col}} \leftarrow \min(T_{\text{col}}, N - j_{\text{blk}})$
    for each row block $i_{\text{blk}} \in [0, N)$ with step T_{row} **do**
      $c_{\text{blk_row}} \leftarrow \min(T_{\text{row}}, N - i_{\text{blk}})$
      Initialize systolic local accumulator tile: $\mathbf{C}_{\text{local}}[0 : T_{\text{row}} - 1, 0 : T_{\text{col}} - 1] \leftarrow 0$
      for each dimension slice $k_0 \in [0, d_k)$ with step 1 **do**
        Skew & stream local Q slice $Q[i_{\text{blk}} : i_{\text{blk}} + T_{\text{row}} - 1, k_0]$ into PE row inputs

Algorithm 3 Stage 2 — Pipelined Dyadic Scaler Unit (`scale_unit`)

Require: Raw Attention Score row vector $\mathbf{s}_{\text{raw}}[i, 0 : N-1] \in \text{INT32}^{1\times N}$;
Dyadic scaling parameters: Mantissa $M \in [0, 2^{18}-1]$, Exponent $E \in [0, 31]$;
Sequence length $N = 256$.
Ensure: Scaled Attention Score row vector $\mathbf{s}_{\text{scaled}}[i, 0 : N-1] \in \text{INT16}^{1\times N}$ saturated to $[-32768, 32767]$.

for each key token index $j \in [0, N)$ **do**
  $P_{\text{mult}} \leftarrow s_{\text{raw}}[i, j] \times M$ ▷ *DSP multiplication: INT32 × INT18 → INT50*
  $S_{\text{shifted}} \leftarrow P_{\text{mult}} \ggg E$ ▷ *Arithmetic right shift by E positions*
  $s_{\text{scaled}}[i, j] \leftarrow \text{Clamp}_{\text{INT16}}(S_{\text{shifted}})$ ▷ *Saturation clamp to* $[-32768, 32767]$
end for
return $\mathbf{s}_{\text{scaled}}[i, 0 : N-1]$

Algorithm 4 Stage 3 — Reconfigurable 3-Pass Hardware IntSoftmax Engine (`softmax_lut`)

Require: Scaled Attention Score row vector $\mathbf{s}_{\text{scaled}}[i, 0 : N-1] \in \text{INT16}^{1\times N}$;
True Dual-Port Exp-LUT BRAM table EXP_BRAM_LUT[0 : 1023] (1024 × 20-bit);
Sequence length $N = 256$.
Ensure: Normalized Attention Probability row vector $\mathbf{P}[i, 0 : N-1] \in \text{UINT16}^{1\times N}$ where $P[i, j] \in [0, 32767]$.

▷ **Pass 1: Streaming Row Maximum Search**
$m_i \leftarrow -32768$ ▷ *Initialize to minimum INT16 value*
for each key token index $j \in [0, N)$ **do**
  $m_i \leftarrow \max(m_i, s_{\text{scaled}}[i, j])$ ▷ *Dynamic peak tracking comparator*
end for

▷ **Pass 2: Exponentiation via BRAM LUT & Denominator Summation**
$S_i \leftarrow 0$ ▷ *Initialize 24-bit denominator accumulator*
for each key token index $j \in [0, N)$ **do**
  $d_{i,j} \leftarrow s_{\text{scaled}}[i, j] - m_i$ ▷ *Subtract max: guaranteed* $d_{i,j} \le 0$
  $\text{lut_addr} \leftarrow \text{Clamp}_{\text{INT}}(-1023, 0, d_{i,j}) + 1023$ ▷ *Bound check to BRAM index* $[0, 1023]$
  $e_{i,j} \leftarrow \text{EXP_BRAM_LUT}[\text{lut_addr}]$ ▷ *Lookup exponential value in UFix20_19*
  $S_i \leftarrow S_i + e_{i,j}$ ▷ *Accumulate sum* $\sum e^{s_j - m_i}$
end for

▷ **Intermediate: 32-Step Restoring Integer Division**
$R_i \leftarrow 32_\text{STEP_RESTORING_DIVIDER}(2^{31}, S_i)$ ▷ *Reciprocal multiplier* $R_i = \lfloor 2^{31}/S_i \rfloor$ *(UFix32_31)*

▷ **Pass 3: Probability Normalization & Saturation**
for each key token index $j \in [0, N)$ **do**
  $M_{\text{prob}} \leftarrow e_{i,j} \times R_i$ ▷ *Multiplier: 20-bit × 32-bit → 52-bit*
  $P[i, j] \leftarrow \text{Clamp}_{\text{UINT16}}(M_{\text{prob}} \ggg 16)$ ▷ *Normalized probability in range* $[0, 32767]$
end for
return $\mathbf{P}[i, 0 : N-1]$

Algorithm 5 Stage 4 — Context Score × Value GEMM Engine (`score_v_gemm`)

Require: Normalized Probability row vector $\mathbf{P}[i, 0 : N-1] \in \text{UINT16}^{1\times N}$;
Value matrix $V \in \text{INT8}^{N\times d_k}$ stored in on-chip Ping-Pong BRAM (V_{MEM});
Constants: Sequence length $N = 256$, Head dimension $d_k \in \{16, 20, 24\}$.
Ensure: Context output token vector $\mathbf{o}_i = O[i, 0 : d_k - 1] \in \text{INT8}^{1\times d_k}$.

for each feature channel $d \in [0, d_k)$ **parallel do**
  $\text{acc}_{\text{out}}[d] \leftarrow 0$ ▷ *Initialize 32-bit channel accumulator*
  for each token index $j \in [0, N)$ **do**
    $\text{acc}_{\text{out}}[d] \leftarrow \text{acc}_{\text{out}}[d] + \big(P[i, j] \times V_{\text{MEM}}[j, d]\big)$ ▷ *UINT16 × INT8 → INT24 accumulation*
  end for
  $S_{\text{proj}} \leftarrow \text{acc}_{\text{out}}[d] \ggg 15$ ▷ *Fixed-point 15-bit shift right*
  $O[i, d] \leftarrow \text{Clamp}_{\text{INT8}}\big(S_{\text{proj}}\big)$ ▷ *Saturation clamp to* $[-128, 127]$
end for
return $O[i, 0 : d_k - 1]$